

12.3 Loading Documents

The annual reports in PDF format contain valuable information about the project and can be found on the WordPress website at <https://viel-falter.at/ergebnisse/veroeffentlichungen>. To avoid having to download each of the 18 PDF documents individually, we've once again made use of AI:

Prompt: How do I save linked PDF documents from the website <https://viel-falter.at/ergebnisse/veroeffentlichungen> to a local folder `pdfs` using Linux shell commands? Don't use temporary files but Unix pipes and remove duplicates. This site uses only absolute URLs.

We've tweaked the prompt a little because *Claude 3.5 Sonnet* initially created temporary files, which we don't consider necessary. Apart from the superfluous `-p` when creating the directory in the first line (the switch would only be necessary if you were creating a nested directory structure), there isn't much to criticize:

```
mkdir -p pdfs && \
wget -qO- https://viel-falter.at/ergebnisse/veroeffentlichungen/\
| grep -oP 'href="[ ^"]*\.\pdf"' | \
sed 's/href="//;s/"$//' | \
sort -u | \
xargs -I {} wget -qnc -P pdfs {}
```

The `wget` command in line 2 loads the HTML code of the web page, which is forwarded to the `grep` command by the pipe character. The `-o` switch is used here to output only the part of the line in which the regular expression was found. The setting for Perl-compatible regular expressions (`-P`) wouldn't be necessary in this case but is also not an error. `sed` is used to remove the preceding `href=` string and the quotation marks from the links. `sort -u` removes the duplicates, and finally `xargs` executes a `wget` command for each link found, with `-P` specifying the path for saving the documents.

Without indicating that all URLs are absolute, Claude generates an address with a preceding protocol and host name (<https://viel-falter.at/>) in the last step, but points out in the explanatory text that this procedure will only work for relative links.

The command used here doesn't necessarily have to work just as well on another website. The search term in the `grep` command uses double quotation marks, which isn't actually required in HTML; however, the WordPress content management system uses double quotation marks.

Other than the PDF documents, we'll also include a list of websites in our embedding model. These are downloaded from the internet and converted into text when the application gets started.

12.3.1 SimpleDirectoryReader and SimpleWebPageReader

The LlamaIndex library uses *documents* and *nodes* as essential components. Documents are containers for data, such as a PDF document or a Markdown document. However, the output of a database query or the response from a web server can also be a document. Metadata is stored for a document (e.g., the file name of a file) and relations to other documents (e.g., the folder on the hard disk). Nodes are parts of a document that also contain metadata and relations.

LlamaIndex makes it very easy for us to convert the downloaded PDF documents into the internal document format:

```
from llama_index.core import SimpleDirectoryReader

pdf_documents = SimpleDirectoryReader("pdfs").load_data(
    show_progress=True
)
```

The `SimpleDirectoryReader` from the `llama_index.core` package generates a list of all files in the `pdfs` folder in the LlamaIndex document format. In our case, these are 18 PDF documents. The `show_progress=True` switch creates progress bars in the command line. As an extension to the example in [Section 12.1](#), we'll also use some important websites as documents. For this purpose, we need the `SimpleWebPageReader` library:

```
from llama_index.readers.web import SimpleWebPageReader

webpages = [
    "https://viel-falter.at/",
    "https://viel-falter.at/monitoring/ziele-vision/",
    "https://viel-falter.at/projektpartner/",
    "https://viel-falter.at/news/",
    "https://viel-falter.at/tagfalter-exkursion/",
];

web_documents = SimpleWebPageReader(
    html_to_text = True).load_data(webpages);
```

The `html_to_text` option specifies that we're only interested in the textual content of the web pages. Both `pdf_documents` and `web_documents` now contain a list of LlamaIndex documents, which we'll convert into a searchable index in a further step. To combine the two lists, we use the `+` operator in Python:

```
documents = pdf_documents + web_documents
```

However, the `SimpleDirectoryReader` and the `SimpleWebPageReader` are just two examples of ways in which you can load data into LlamaIndex. On the LLamaHub (<https://llamahub.ai/>) website, you'll find Python modules for countless other formats.